



Aula 09 - ORM Parte 1: Relacionamentos com o Eloquent

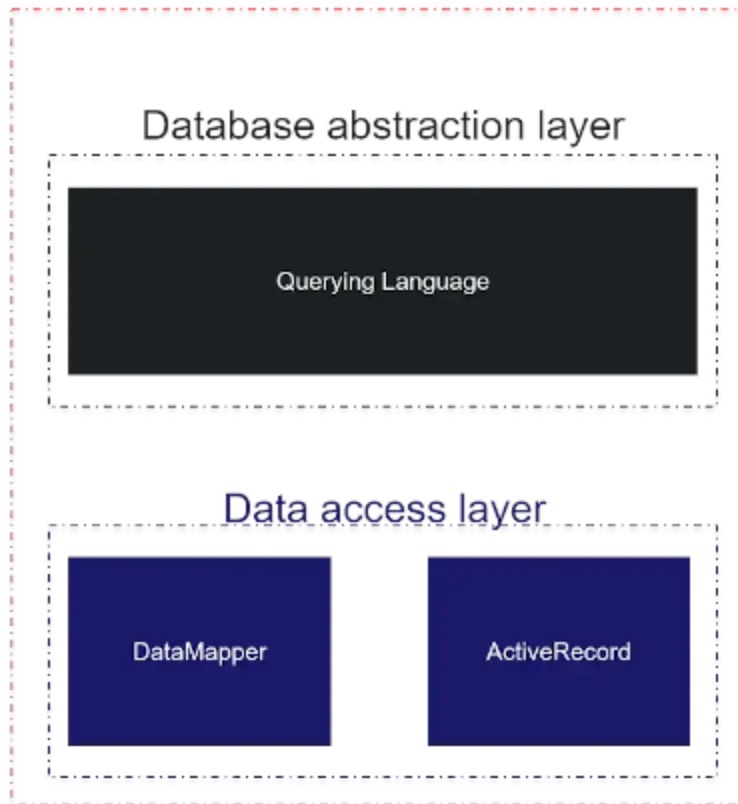
IFSUL | CSTSI | Prof. Gill Velleda Gonzales

LICENÇA: Creative Commons — Attribution-ShareAlike 4.0 International
— CC BY-SA 4.0

Nesta aula veremos brevemente as diferentes estratégias de ORM existentes e qual o Eloquent utiliza. Vamos explorar também relações diretas e indiretas de diferentes níveis através dos métodos do **Eloquent** e alguns pacotes extras.

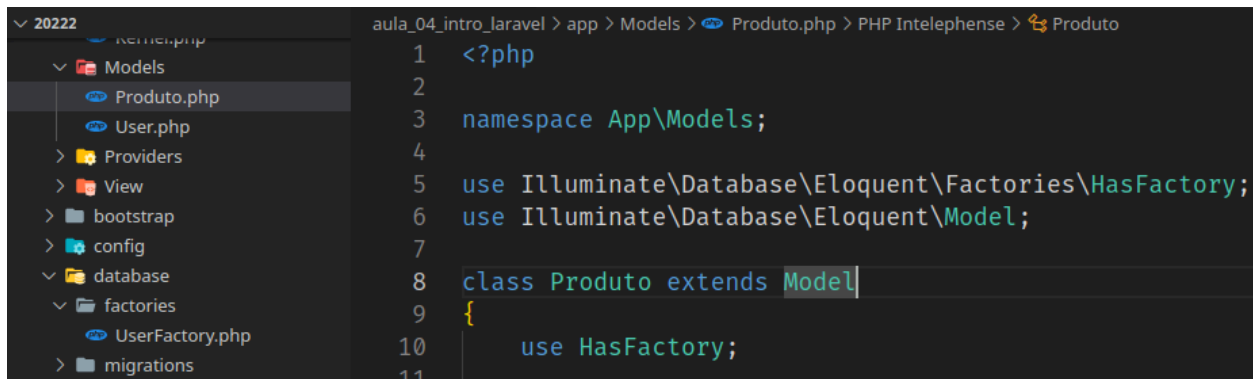
Padrão Active Records versus Data Mapper:

ORM



Fonte: <https://medium.com/oceanize-geeks/the-active-record-and-data-mappers-of-orm-pattern-eefb8262b7bb>

O padrão de Mapeamento Objeto Relacional (**ORM**) do Eloquent segue a estratégia conhecida como **Active Records**, onde classes que representam os Modelos da aplicação, com suas regras de negócios, também herdam os métodos e propriedades necessárias para a persistência dos objetos no banco de dados. Por esta razão, uma Model no Laravel, estende diretamente a classe Model da camada do Eloquent:



```
1 <?php
2
3 namespace App\Models;
4
5 use Illuminate\Database\Eloquent\Factories\HasFactory;
6 use Illuminate\Database\Eloquent\Model;
7
8 class Produto extends Model
9 {
10     use HasFactory;
11 }
```

Desta forma, os dados a serem persistidos no banco, assim como os relacionamentos são declarados na própria classe do modelo. Um exemplo é a propriedade **\$fillable** que define quais propriedades devem ser persistidas, garantindo proteção a **atribuição massiva**.

```
1 class Produto extends Model
2 {
3     use HasFactory;
4
5     protected $fillable = [
6         "nome",
7         "descricao",
8         "qtd_estoque",
9         "preco",
10        "importado"
11    ];
12 }
```

Existem outras estratégias ORM, como o **Data Mapper**, utilizada pelo **Doctrine**, e outros (Hibernate no Spring). A grande diferença entre esta estratégia e aquela aplicada pelo Eloquent, ou outros como RubyOnRails, está na implementação da lógica do modelo de negócios separada da camada de acesso aos dados.

Neste caso existem estratégias para mapear os atributos com as colunas, assim como os relacionamentos existentes. Esta estratégia varia de sintaxe específica,

anotações no código, ou arquivos externos de configuração, XML ou YAML

Exemplo de mapeamento de dados no **Doctrine ORM**:

```
ATTRIBUTE ANNOTATION XML
1  <?php
2  // src/Product.php
3
4  use Doctrine\ORM\Mapping as ORM;
5
6  #[ORM\Entity]
7  #[ORM\Table(name: 'products')]
8  class Product
9  {
10     #[ORM\Id]
11     #[ORM\Column(type: 'integer')]
12     #[ORM\GeneratedValue]
13     private int|null $id = null;
14     #[ORM\Column(type: 'string')]
15     private string $name;
16
17     // .. (other code)
18 }
```

Exemplo de uso de Atributos do PHP para relacionar propriedades da classe a entidade no banco de dados. [Link Referência](#).

É possível usar Doctrine no Laravel e para isso existe pacote específico, o Laravel Doctrine:

Installation in Laravel 6+

Install this package with composer: composer require laravel-doctrine/orm Thanks to Laravel auto package discovery feature, the ServiceProvider and Facades are automatically
<http://www.laraveldoctrine.org/docs/1.8/orm/installation>



No entanto, o foco da disciplina será o ORM padrão Eloquent.

Relacionamentos com o Eloquent ORM:

O Eloquent possui vários métodos para representar os relacionamentos entre diferentes modelos. Estes métodos variam para cada tipo de relacionamento entre os modelos, veremos os mais comuns:

- **hasOne**(**Model::class**): define que a classe model onde foi implementado esse relacionamento possui um relacionamento **1:1** com o modelo da classe passada como parâmetro (**Laravel Docs**).
- **hasMany**(**Model::class**): define a relação **1:N**, ou seja, o modelo possui vários outros modelos do Model passado como parâmetro (**Laravel Docs**);
- **belongsTo**(**Model::class**): usado para representar as relações **1:1** e **1:N**, no segundo modelo da relação, ou seja, é o registro inverso do **hasOne** ou **hasMany** (**Laravel Docs**)
- **belongsToMany**(**Model::class**): Usado para representar a relação **N:N**, muitos para muitos, aplica-se em ambas as classes Model (**Laravel Docs**).
- **Documentação** com definição de todos os relacionamentos suportados!

Instalação do Banco com Regiões, Estados, Fornecedores e Produtos

O nosso banco possui relações (1:N) entre as seguintes tabelas:

Região→Estado→Fornecedores→Produtos

Vamos criar a camada de Modelo deste banco com suas respectivas migrations, factories e seeders.

Modelo de Região:

Criação da model Região com **migration** (m) e **seeder** (s)

```
php artisan make:model Regiao -ms
```

Este comando do **artisan** criará os seguintes arquivos:

- **/app/Models/Regiao.php**
- **/database/migrations/2022_08_28_041107_create_regioes_table.php**
- **/database/seeders/RegiaoSeeder.php**

```
...laravel/cstsi_daoo_20222_php_poo/aula_04_intro_laravel <?php 8.1.13 aula-08-
• $ php artisan make:model Regiao -ms

INFO Model [app/Models/Regiao.php] created successfully.

INFO Created migration [2022_12_13_195054_create_regiaos_table].

INFO Seeder [database/seeder/RegiaoSeeder.php] created successfully.
```

Classe Models/Regiao:

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Regiao extends Model
{
    use HasFactory;

    protected $table = 'regioes';
    protected $fillable = ['nome'];

    public function estados(){
        return $this->hasMany(Estado::class);
    }
}
```

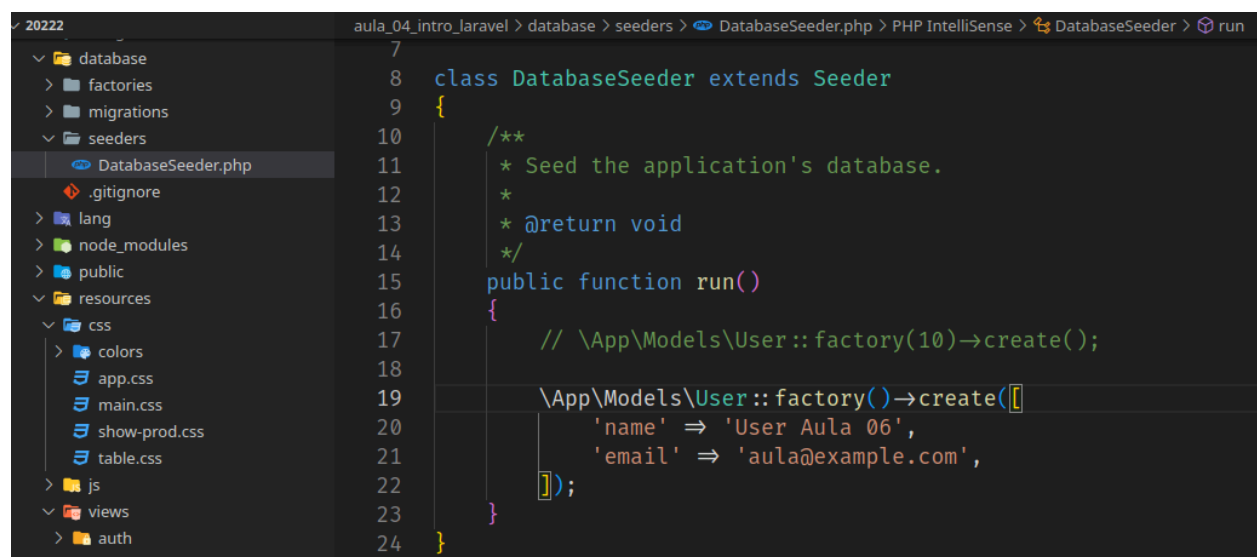
Migration Regiao

```
public function up()
{
    Schema::create('regioes', function (Blueprint $table) {
        $table->id();
        $table->string('nome', 50);
        $table->timestamps();
    });
}
```

```
});  
}
```

Seeders

Seeders no Laravel são classes responsáveis por popular o nosso banco de dados com dados para teste ou dados reais. Todos os seeders são localizados na pasta **database/seeders/** e podem ser executados através da classe **DatabaseSeeder**, pré-configurada pelo framework. Já usamos esta classe para criar um usuário para o login do Breeze.



```
7  
8 class DatabaseSeeder extends Seeder  
9 {  
10     /**  
11      * Seed the application's database.  
12      *  
13      * @return void  
14      */  
15     public function run()  
16     {  
17         // \App\Models\User::factory(10)→create();  
18  
19         \App\Models\User::factory()→create([  
20             'name' => 'User Aula 06',  
21             'email' => 'aula@example.com',  
22         ]);  
23     }  
24 }
```

No entanto podemos utilizar para outros modelos além do User. Nos próximos exemplos utilizaremos seeders para popular o banco com dados reais de Regiões e Estados Brasileiros, e depois com dados fakes para Fornecedores e Produtos.

Seeder de Regiao

Criaremos o método **run** do **seeder** de Região **/database/seeders/RegiaoSeeder** para a importação das regiões a partir de um arquivo JSON localizado em:

<https://github.com/chandez/Estados-Cidades-IBGE/>

```

<?php

namespace Database\Seeders;

use Illuminate\Database\Console\Seeds\WithoutModelEvents;
use Illuminate\Database\Seeder;

use App\Models\Regiao;
use Exception;
use Illuminate\Support\Facades\Http;
use Illuminate\Support\Facades\Log;

class RegiaoSeeder extends Seeder
{
    /**
     * Run the database seeds.
     */
    public function run(): void
    {
        try {
            if (Regiao::all()→count()) {
                Log::channel('stderr')→info("O banco já possui regioes!");
                print_r(Regiao::all()→pluck('nome','id'));
                return;
            }

            $jsonUrl = 'https://raw.githubusercontent.com/chandez/Estados-Cidades-IBGE/master/json/regioes.json';
            $response = Http::get($jsonUrl);
            $regioes = $response→json()['data'];

            if (!$regioes)
                throw new \Exception("Erro ao carregar arquivo JSON de Regioes!\nURL:$jsonUrl");
        }
    }
}

```

```

$listRegioes = [];
foreach ($regioes as $regiao)
    $listRegioes[] = ["nome" => $regiao['Nome']];

if (!Regiao::insert($listRegioes))
    throw new \Exception("Erro ao inserir Regioes!", 1);

Log::channel('stderr')->info("Regiões inseridas com sucesso!");
print_r(Regiao::all()->pluck('nome','id'));
} catch (Exception $error) {
    throw new Exception("Erro ao processar o seed de Regioes!\n {$error-
>getMessage()}");
}
}
}

```

Modelo Estado:

Criação da Model de Estados:

```
php artisan make:model Estado -ms
```

Este comando irá criar a Model Estado e os arquivos de migration e seeder:

```

...laravel/cstsi_daoo_20222_php_poo/aula_04_intro_laravel <?php 8.1.13 aula-08-01
• $ php artisan make:model Estado -ms

INFO Model [app/Models/Estado.php] created successfully.
INFO Created migration [2022_12_13_195954_create_estados_table].
INFO Seeder [database/seeder/EstadoSeeder.php] created successfully.

```

Código da Classe Estado:

```

<?php

namespace App\Models;

```

```

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Estado extends Model
{
    use HasFactory;

    protected $fillable = ['nome','uf','codigouf','regiao_id'];

    public function regiao(){
        return $this->belongsTo(Regiao::class);
    }
}

```

Já incluímos na Model de Estado a relação com a Model Região, o mesmo é feito na Model de Região porém usando o método **hasMany**:

Migration Estado

```

public function up()
{
    Schema::create('estados', function (Blueprint $table) {
        $table->id();
        $table->foreignId('regiao_id')
            ->references('id')->on('regioes')
            ->cascadeOnDelete();
        $table->integer('codigouf');
        $table->string('nome', 50);
        $table->char('uf', 2);
        $table->timestamps();
    });
}

```

Observação: neste exemplo especificamos a tabela da chave estrangeira pois não usamos as convenções de nomes do Laravel, quando usadas é possível substituir o código:

```
$table->foreignId('regiao_id')
    ->references('id')->on('regioes')
    ->cascadeOnDelete();
```

Forma simplificada quando se usa a convenção de nomes:

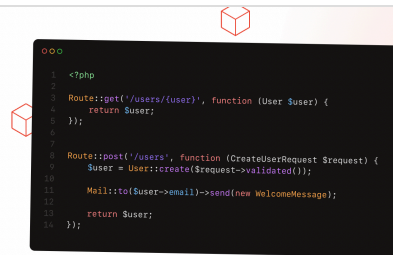
```
$table->foreignId('regiao_id')
    ->constrained()
    ->cascadeOnDelete();
```

No entanto esta simplificação só funcionaria se o nome da tabela de regiões fosse *"regiao"*

Laravel - The PHP Framework For Web Artisans

Migrations are like version control for your database, allowing your team to define and share the application's database schema definition. If you have ever had to tell a teammate to

 <https://laravel.com/docs/9.x/migrations#foreign-key-constraints>



```
1 <?php
2
3 Route::get('/users/{user}', function (User $user) {
4     return $user;
5 });
6
7
8 Route::post('/users', function (CreateUserRequest $request) {
9     $user = User::create($request->validated());
10
11     Mail::to($user->email)->send(new WelcomeMessage);
12
13     return $user;
14 });
```

Criação da classe EstadoSeeder

Nesta classe também usaremos o repositório com os nomes de todos os estados.

```
<?php
```

```
namespace Database\Seeders;
```

```
use Illuminate\Database\Console\Seeds\WithoutModelEvents;
```

```
use App\Models\Estado;
```

```
use Illuminate\Database\Seeder;
```

```
use Illuminate\Support\Facades\Http;
```

```
use Illuminate\Support\Facades\Log;
```

```
class EstadoSeeder extends Seeder
{
```

```

/**
 * Run the database seeds.
 *
 * @return void
 */
public function run()
{
    try {
        if (Estado::all()→count()) {
            Log::channel('stderr')→info("O banco já possui estados cadastrado
s!");
            print_r(Estado::all()→pluck('nome','id'));
            return;
        }

        $jsonUrl = 'https://raw.githubusercontent.com/chandez/Estados-Cidades-IBGE/master/json/estados.json';
        $response = Http::get($jsonUrl);
        $estados = $response→json()['data'];

        if (!$estados)
            throw new \Exception("Erro ao carregar arquivo JSON de Estados!\n
URL:$jsonUrl");

        $listEstados = [];
        foreach ($estados as $estado)
            $listEstados[] = [
                "nome"    ⇒ $estado['Nome'],
                "codigouf" ⇒ $estado["CodigoUf"],
                "uf"       ⇒ $estado["Uf"],
                "regiao_id" ⇒ $estado["Regiao"]
            ];

        if (!Estado::insert($listEstados))
            throw new \Exception("Erro ao inserir estados!", 1);
    }
}

```

```

        Log::channel('stderr')→info("Estados inseridos com sucesso!");
        print_r(Estado::all()→pluck('uf','id'));
    } catch (\Exception $error) {
        throw new \Exception("Erro ao processar o seed de estados!\n {$error
→getMessage()}");
    }
}
}
}

```

Relação Região → Estados

```

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Regiao extends Model
{
    use HasFactory;

    protected $table = 'regioes';
    protected $fillable = ['nome'];

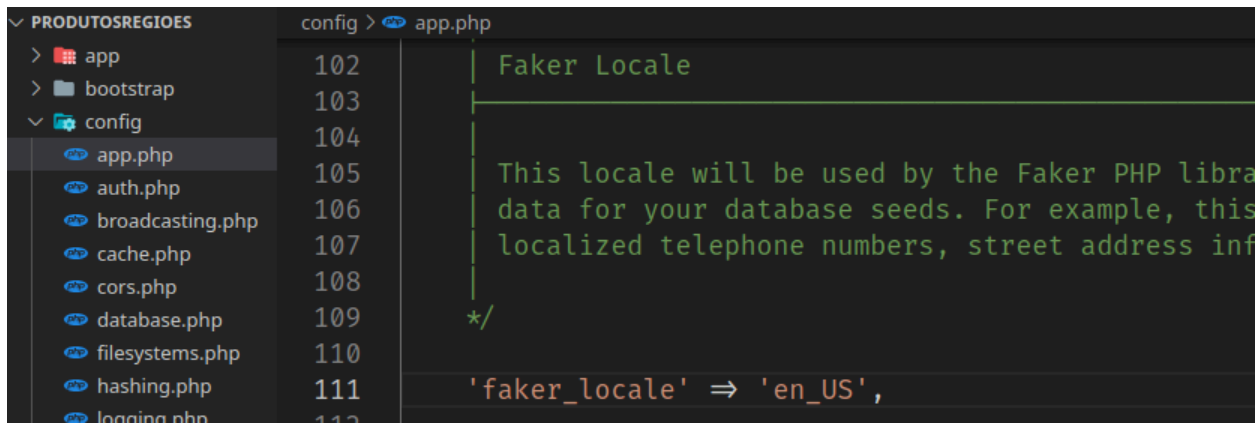
    public function estados(){
        return $this→hasMany(Estado::class);
    }
}

```

Note que como uma Região possui vários Estados, o nome do método ficou no plural (**estados**). O contrário acontece na Model de Estado, o nome fica no singular, visto que o Estado só pode pertencer a uma Região, ou seja, método **regiao**.

Factories

Para os próximos modelos iremos utilizar a biblioteca ***faker*** para construir os ***factories*** de cada modelo. ***Factory*** nada mais é do que um padrão utilizado para criar objetos dada uma configuração como template, um molde. No Laravel utilizamos factories para popular (preencher) o banco de dados com dados de exemplo, ou teste. Podemos gerar dados inclusive em português, para tanto iremos alterar o ***locale*** (localização) da lib ***faker*** na opção ***faker_locale*** do arquivo ***/config/app.php*** e sobrescrever com o valor ***"pt_BR"***.



Após alterada ficará desta forma:

```
109 | */
110 |
111 | // 'faker_locale' => 'en_US',
112 | 'faker_locale' => 'pt_BR',
113 |
114 | /*
```

Agora estamos prontos para gerar os Factories de ***Fornecedores*** e ***Produtos***, os próximos modelos do nosso banco de dados de exemplo.

Modelo Fornecedor:

- Comando artisan para a criação da model Fornecedor

```
php artisan make:model Fornecedor -mf
```

Criaremos uma **migration (m)** e um **factory (f)** junto ao modelo Fornecedor.

Criação da migration de Fornecedor:

```
public function up()
{
    Schema::create('fornecedores', function (Blueprint $table) {
        $table->id();
        $table->string('nome');
        $table->text('endereco');
        $table->string('cnpj');
        $table->string('telefone');
        $table->string('email');
        $table->foreignId('estado_id')->constrained()->cascadeOnDelete();
        $table->timestamps();
    });
}
```

Documentação sobre os tipos de colunas suportadas: [Link](#)

Criação do factory para fornecedores:

```
return [
    "nome" => fake()->company(),
    "cnpj" => fake()->cnpj(),
    "email" => fake()->companyEmail(),
    "estado_id" => fake()->numberBetween(1, 27),
    "telefone" => fake()->cellphoneNumber(),
    "endereco" => fake()->sentence(3) . ' nr:' . fake()->randomNumber(4, f
else)
];
```

Classe Model Fornecedor

```
<?php
```

```

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Fornecedor extends Model
{
    use HasFactory;

    protected $table = 'fornecedores';
    protected $fillable = [
        "nome",
        "cnpj",
        "email",
        "estado_id",
        "telefone",
        "endereco"
    ];

    public function estado()
    {
        return $this->belongsTo(Estado::class);
    }

    public function produtos()
    {
        return $this->hasMany(Produto::class);
    }
}

```

Modelo Produtos:

Criação da Model de Produtos:

```
php artisan make:model Produto -mf
```

Criação da migration de Produtos

```
public function up()
{
    Schema::create('produtos', function (Blueprint $table) {
        $table->id();
        $table->foreignId('fornecedor_id')
            ->references('id')->on('fornecedores')
            ->cascadeOnDelete();
        $table->string('nome');
        $table->string('descricao');
        $table->float('preco');
        $table->integer('qtd_estoque');
        $table->boolean('importado')->default(false);
        $table->timestamps();
    });
}
```

Factory para produtos:

```
return [
    "nome" => fake()->text(30),
    "descricao" => fake()->sentence(10),
    "preco" => fake()->randomFloat(2, 100, 10000),
    "qtd_estoque" => fake()->randomNumber(3, false),
    "importado" => fake()->numberBetween(0, 1)
];
```

Adicionamos a relação de **Produto** com **Fornecedor** na model **Produto**
(Fornecedor 1:N Produto)

Usamos neste caso o método **belongsTo** para sinalizar ao Eloquent que Produto pertence a um fornecedor.

```

class Produto extends Model
{
    use HasFactory;

    protected $fillable = [
        "nome",
        "descricao",
        "preco",
        "qtd_estoque",
        "importado",
        "fornecedor_id"
    ];

    public function fornecedor(){
        return $this->belongsTo(Fornecedor::class);
    }
}

```

Atualizamos também a model **Fornecedor** com a relação com **Produto**, neste caso aplicamos o método **hasMany**, indicando que Fornecedor possui mais de um Produto. Note a relação de Fornecedor com Estado, neste caso é o Fornecedor que pertence a um estado, portanto usamos o método **belongsTo**.

```

class Fornecedor extends Model
{
    use HasFactory;

    protected $table = 'fornecedores';
    protected $fillable = [
        "nome",
        "cnpj",
        "email",
        "estado_id",
        "telefone",
        "endereco",
    ];
}

```

```
];

public function estado(){
    return $this->belongsTo(Estado::class);
}

public function produtos(){
    return $this->hasMany(Produto::class);
}
}
```

Atualizando o **databaseSeeder** para utilizar os **seeders** e **factories** já configurados:

```
public function run()
{
    // \App\Models\User::factory(10)→create();

    \App\Models\User::factory()→create([
        'name' => 'Test User',
        'email' => 'test@example.com',
    ]);

    $seedRegiao = new RegiaoSeeder();
    $seedRegiao→run();

    (new EstadoSeeder)→run();

    \App\Models\Fornecedor::factory(fake()→randomNumber(2))
        →hasProdutos(fake()→randomNumber(1))
        →create();
}
```

Executamos nossos seeders para popular o banco de dados:

```
php artisan migrate:fresh
php artisan db:seed
```

Vamos usar o <https://github.com/laravel/tinker> para rodar as queries dos próximos exemplos, basta rodar o comando a seguir em um novo terminal:

```
php artisan tinker
```

Obs: o tinker é uma ferramenta de REPL para o Laravel, **Read-Eval-Print-Loop**.

Consultas com Eloquent:

Antes de começarmos veremos um exemplo para entender como o **Eloquent** monta as suas consultas. Internamente o **Eloquent** utiliza a classe **Query/Builder** que podemos acessar a partir da class **BD**, veja o exemplo:

```
DB::Table('produtos')->join('fornecedores','fornecedor_id','=', 'fornecedores.id')->get()->first()
```

```
>>> DB::Table('produtos')->join('fornecedores','fornecedor_id','=', 'fornecedores.id')->get()->first()
=> {#4901
  +"id": 1,
  +"fornecedor_id": 1,
  +"nome": "Soto e Queirós S.A.",
  +"descricao": "Architecto reprehenderit velit blanditiis voluptatibus aut magnam ipsum beatae par
tia.",
  +"preco": 4704.0,
  +"qtd_estoque": 588,
  +"importado": 0,
  +"created_at": "2022-09-15 20:19:08",
  +"updated_at": "2022-09-15 20:19:08",
  +"endereco": "Et molestiae aut laudantium. nr:672",
  +"cnpj": "21.726.966/0001-80",
  +"telefone": "(49) 97162-6762",
  +"email": "sara.cruz@rios.org",
  +"estado_id": 13,
}
```

Com o método `get()` é retornada um Coleção (**Illuminate/Support/Collection**) e dela chamamos o método `first()`, e assim recuperar o primeiro resultado.

No entanto no exemplo acima, tanto a tabela produto como a tabela fornecedores possuem o campo nome, portanto, para diferenciarmos corretamente, no próximo

exemplos utilizaremos o método ***select***:

```
DB::Table('produtos')->join('fornecedores','fornecedor_id','=', 'fornecedores.id')->select('*', 'produtos.nome as nome', 'fornecedores.nome as nome_fornecedor')->get()->first()
```

```
>>> DB::Table('produtos')->join('fornecedores','fornecedor_id','=', 'fornecedores.id')->select('*', 'produtos.nome as nome', 'fornecedores.nome as nome_fornecedor')->get()->first()
=> {#4898
    +"id": 1,
    +"fornecedor_id": 1,
    +"nome": "Ipsum consectetur est quasi.",
    +"descricao": "Architecto reprehenderit velit blanditiis voluptatibus aut magnam ipsum beatae pariatur aut a sit molestiae.",
    +"preco": 4704.0,
    +"qtd_estoque": 588,
    +"importado": 0,
    +"created_at": "2022-09-15 20:19:08",
    +"updated_at": "2022-09-15 20:19:08",
    +"endereco": "Et molestiae aut laudantium. nr:672",
    +"cnpj": "21.726.966/0001-80",
    +"telefone": "(49) 97162-6762",
    +"email": "sara.cruz@rios.org",
    +"estado_id": 13,
    +"nome_fornecedor": "Soto e Queirós S.A.",
}
```

Observe agora o atributo ***nome_fornecedor*** no objeto retornado.

Nos exemplos acima, acessamos o primeiro objeto na coleção de resultados pelo método `first()`. A coleção de resultados nada mais é do que uma abstração criada acima de um Array de Objetos, mas que nos traz diversos ***métodos*** para manipular os dados de uma maneira mais simples, como por exemplo o método ***map*** abaixo:

```
DB::Table('produtos')->join('fornecedores','fornecedor_id','=', 'fornecedores.id')->get()->map(fn($i) => $i->nome)
```

O exemplo acima, o ***map*** está iterando cada elemento da coleção e retornando um novo array filtrando apenas no atributo nome do objeto, a sintaxe `fn() => expressao` é a definição de uma ***Arrow Function*** no php, mas poderia ser passada uma função anônima como `function($nome){return $i->nome}`

```
DB::Table('produtos')->join('fornecedores','fornecedor_id','=', 'fornecedores.id')->select('*', 'produtos.nome as nome', 'fornecedores.nome as nome_fornecedor')
```

```
dor')→get()→map(fn($p)⇒[$p→nome_fornecedor,$p→nome,$p→preco])
```

No próximo exemplo usaremos a cláusula `where` com o operador `between` (`whereBetween`) e depois ordenar com o método `orderBy`:

```
DB::Table('produtos')→whereBetween('preco',[100,500])→orderBy('preco','desc')→get()→first()
```

Consulta com Query/Builder

Na classe **Query/Builder** temos vários métodos para montar consultas, como os exemplos acima.

- **SELECT**: Seleciona as colunas que retornarão ao usar o Query/Builder.
- **WHERE**: Monta cláusulas *where*, aceita a passagem do campo, do operador e do valor. Pode ser encadeado, gerando a lógica de AND. Existe também o **orWhere** se precisar encadear com a lógica de OR.

- Encadeamento:

```
DB::Table('produtos')  
    →where('importado',true)  
    →whereBetween('preco',[100,500])  
    →orWhere('qtd_estoque',>,100)
```

- Variações: **whereBetween**, **whereIn**, **whereNot** entre outras.
- **ORDERBY**: Monta a query com o comando ORDERBY, recebe a direção como parametro (DESC|ASC)
 - Variações: **orderByDesc**, **latest**, **oldest**, e outras.
- **JOIN**: Aplica as junções entre as tabelas, mas é preciso passar como parâmetros os campos do ON. Possui também as **variações**.
- **LIMIT**: Aplica a cláusula para limitar os resultados.
- E existem diversos outros métodos para construir o SQL, veja na **documentação** ou na **referência da API**.

- **GET:** executa a query retornando uma Support/Collection com os resultados.

Consulta com Eloquent/Builder

O Eloquent/Builder também implementa métodos semelhantes ao Query/Builder, mas tem algumas variações devido a classe trabalhar diretamente com os modelos do Eloquent, possuindo métodos que não existem no Query/Builder, como:

- **firstWhere:** Retorna a primeira model que combinar com a cláusula where.
- **select:** não possui, visto que o Builder está ligado ao Model do Eloquent.
- **with:** Define as relações que devem ser carregadas na consulta
- **get:** toda vez que montamos queries com o Eloquent/Builder precisamos chamar o método get ao final para executar a query e retornar os resultados, que serão uma coleção do tipo Eloquent/Collection.

Consulta com Eloquent/Model

Alguns métodos para consultar dados dos modelos estão diretamente implementados na classe Eloquent/Model, como:

- **load:** Lista o modelo com seus relacionamentos de acordo com o parâmetro passado.
- **all:** retorna todos os modelos.
- **Relações:** na model também teremos todos os métodos que representam os relacionamentos.

As consultas equivalentes utilizando o **Eloquent** seriam:

```
DB::Table('produtos')->join('fornecedores','fornecedor_id','=', 'fornecedores.i  
d')->select('*', 'produtos.nome as nome', 'fornecedores.nome as nome_fornece  
dor')->get()->first()
```

```
Produto::with('fornecedor')->first()
```

```
>>> Produto::with('fornecedor')->first()
[!] Aliasing 'Produto' to 'App\Models\Produto' for this Tinker session.
=> App\Models\Produto {#4868
    id: 2,
    fornecedor_id: 1,
    nome: "Ipsum consectetur est quasi.",
    descricao: "Architecto reprehenderit velit blanditiis voluptatibus au
e.",
    preco: 4704.0,
    qtd_estoque: 588,
    importado: 0,
    created_at: "2022-09-15 20:19:08",
    updated_at: "2022-12-13 16:40:43",
    fornecedor: App\Models\Fornecedor {#4888
        id: 1,
        nome: "Soto e Queirós S.A.",
        endereco: "Et molestiae aut laudantium. nr:672",
        cnpj: "21.726.966/0001-80",
        telefone: "(49) 97162-6762",
        email: "sara.cruz@rios.org",
        estado_id: 13,
        created_at: "2022-09-15 20:19:08",
        updated_at: "2022-09-15 20:19:08",
    },
}
```

```
Produto::whereBetween('preco',[100,500])>orderBy('preco','desc')>get()->fi
rst()
```

Portanto as classes do **Eloquent**, como a Model de Produto, possuem acesso aos métodos do **Query/Builder**, e podemos utilizá-los para montar consultas. No entanto, estas sempre retornaram objetos do **Eloquent**, ou **EloquentCollection**, enquanto que o uso direto do **Query/Builder** retorna ou uma Collection genérica ou um Objeto da **StdClass**, classe padrão do PHP.

Veja o tipo retornado com o **var_dump**:

Query/Builder: retorna um Collection genérica com produtos de preco entre 100 e 500

```
DB::Table('produtos')>whereBetween('preco',[100,500])>orderBy('preco','de
sc')>get();
```

Com maior preco entre 100 e 500 **Query/Builder**: retorna um objeto **StdClass**

```
DB::Table('produtos')→whereBetween('preco',[100,500])→orderBy('preco','desc')→get()→first();
```

Eloquent: retorna um objeto da model Produto

```
Produto::whereBetween('preco',[100,500])→orderBy('preco','desc')→get()→first();
```

- Ativando o log das queries:

```
\DB::enableQueryLog(); //Abilita o Log
\DB::getQueryLog(); //Recupera o Log
```

Com a ativação dos logs (*enableQueryLog();*) podemos consultar as últimas queries que rodaram no banco, sendo útil para entender como os métodos do Eloquent constroem através do Query/Builder as consultas SQL.

Também podemos chamar o método toSql() a partir do objeto do Query/Builder

Selecionando Fornecedores de um determinado estado:

```
Fornecedor::whereHas('estado',fn($q)⇒$q→where('uf','like','RS'))→orderBy('nome')→get();
```

Carregando dados do relacionamento:

WITH:

```
Produto::with('fornecedor')→find(1);
```

LOAD:

```
Produto::find(1)→load('fornecedor');
```

Exemplo de seleção de Fornecedores com dados do seu Estado

```

$forneecedorEstados = Forneecedor::with('estado')
    →orderBy('estado_id')
    →whereBetween('estado_id',[10,20])→get()

foreach($forneecedorEstados as $forneecedor)
    print_r([$forneecedor→nome,$forneecedor→estado→uf]);

```

- Filtrando estados da região central ordenados pelo UF em ordem descendente
 - PHP:

```

Estado::whereHas('regiao',fn($q)⇒$q→where('nome','like','%ce%'))→orderBy(
    ('uf','desc'))→get()

```

- Filtrando estados da região sul e sudeste ordenados em ordem descendente por UF

```

Regiao::where('nome','like','%su%')→with(['estados'⇒fn($q)⇒$q→orderBy('u
f','desc')])→get()

```

- Listando Fornecedores de um determinado Estado com seus Produtos:

Eloquent:

```

Fornecedor::whereHas('estado',fn($q)⇒$q→where('uf','RS'))→with('estado','p
rodutos')→get()
//replica o objeto de estado

```

- Contando quantos produtos existem para um determinado Estado:

Eloquent:

```

Produto::whereHas('fornecedor',fn($q)⇒$q→where('estado_id',21))→count();

```

SQL:

```
SELECT COUNT(produtos.id) FROM produtos INNER JOIN fornecedores ON produtos.fornecedor_id = fornecedores.id WHERE fornecedores.estado_id = 21;
```

Mesmo exemplo passando apenas o UF do Estado

```
Produto::whereHas('fornecedor',fn($q)⇒$q→whereHas('estado',fn($q)⇒$q→where('uf','RS'))))→count();
```

SQL: QueryLog

```
select count(*) as aggregate from `produtos` where exists (select * from `fornecedores` where `produtos`.`fornecedor_id` = `fornecedores`.`id` and exists (select * from `estados` where `fornecedores`.`estado_id` = `estados`.`id` and `uf` = 'RS'));
```

Filtrar produtos importados e com preço entre 100 e 2k que sejam de fornecedores da região Sul:

```
Produto::where('importado',1)
    →whereBetween('preco',[100,2000])
    →whereHas('fornecedor',
        fn($q)⇒
            $q→whereHas('estado',fn($q)⇒
                $q→whereHas('regiao',
                    fn($q)⇒
                        $q→where('nome','like','Sul')))))
    →get()
```

```
Produto::where('importado',1)→whereBetween('preco',[100,2000])→whereHas('fornecedor',fn($q)⇒$q→whereHas('estado',fn($q)⇒$q→whereHas('regiao',fn($q)⇒$q→where('nome','like','Sul')))))→get()
```

SQL: QueryLoq

```
select * from `produtos` where `importado` = 1 and `preco` between 1 and 2000 and exists (select * from `fornecedores` where `produtos`.`fornecedor_id` = `fornecedores`.`id` and exists (select * from `estados` where `fornecedores`.`estado_id` = `estados`.`id` and exists (select * from `regioes` where `estados`.`regiao_id` = `regioes`.`id` and `nome` like 'Sul'))))
```

SQL com INNER JOIN:

```
SELECT * FROM produtos INNER JOIN fornecedores ON produtos.fornecedor_id = fornecedores.id INNER JOIN estados ON estados.id = fornecedores.estado_id INNER JOIN regioes ON regioes.id = estados.regiao_id WHERE produtos.importado = 1 and produtos.preco between 1 and 2000 and regioes.nome like 'Sul'
```

Referências:

- ActiveRecord: <https://dev.to/shahershamroukh/understanding-active-record-3lc>
- Active vs Mapper: <https://medium.com/@matheusflauzino/entenda-orm-active-record-e-data-mapper-9be60da0e799>
- Eloquent: <https://laravel.com/docs/5.2/eloquent-relationships>
- Faker: https://fakerphp.github.io/locales/pt_PT/
- Relações de vários níveis:
 - <https://laravel.com/docs/9.x/eloquent>
 - <https://laravel.com/docs/9.x/queries>
 - <https://github.com/staudenmeir>
- Debugging Queries:

- <https://write.corbpie.com/how-to-debug-and-view-a-laravel-db-query/>
- <https://laraveldaily.com/post/laravel-eloquent-tools-debug-slow-sql-queries>
- <https://www.youtube.com/watch?v=SR3RzIfeoZl>
- <https://laravel.com/docs/9.x/telescope>
- N+1 Problem
 - <https://laravel-news.com/laravel-n1-query-problems>
- Scopes
 - <https://laravel-news.com/query-scopes>